M1 Informatique - HAI818I - Modularité et Réutilisation

Année 2024-2025, Session 1, mai 2025.

C. Dony, I. Mougenot, C. Nebut, F. Vargas-Rojas

Durée : 2h00. **Documents non autorisés**. La précision et la concision des réponses sont évaluées positivement.

La partie I porte sur les 4 premières séances de cours (C. Dony, C. Nebut) et la partie II sur les suivantes (I. Mougenot, F. Vargas-Rojas). Les parties sont indépendantes. Chaque partie vaut 10 points.
Merci de rédiger les réponses aux parties A et B de façon séparée (feuilles différentes).

PARTIE I (10 points et 1 heure de temps estimé)

A Schémas de Réutilisation - (environ 3 points)

Considérons la hiérarchie de classes des éléments de stockage *(Directory, File, Link)*, architecturée selon le schéma *Composite*, et dotée d'un mécanisme de visite, selon le schéma *Visiteur*, étudié dans le TD/TP no 1. Vous avez réalisé en TP plusieurs visiteurs dont `RazVisitor`. Soit (ci-dessous) le code de la classe `Visitor` et le code (incomplet) de la classe `Directory` (ou réperoire ou dossier) dans lequel l'attribut `elements` sert à stocker les éléments contenus dans un *directory* (répertoire).

```
1  public abstract class Visitor {
2      public void visitFile(File f){}
3      public void visitDirectory(Directory f){}
4      public void visitLink(Link f) {} }

6  public class Directory extends ElementStockage{
7       protected Collection<ElementStockage> elements ;
8       public void accept (Visitor v){
9         for (ElementStockage s : elements){ s.accept(v) ;}
10        v.visitDirectory(this);} }
```

Listing 1 – Eléments de mise en place du schéma Visiteur sur les éléments de stockage.

Questions

1. Vous savez qu'il est possible de donner le même nom à différents répertoires dans la même arborescence.
   Donnez le code d'une classe `FindDirVisitor` telle que, si `d` est un répertoire et si `v` est une instance de cette classe créée par l'expression `v = FindDirVisitor(''src'');`, alors après l'exécution de `d.accept(v)`, l'expression `v.getFind()` rend la collection des adresses absolues de tous les répertoires dont le nom est `''src''` qui sont sous-répertoire de `d`. On suppose que la méthode `getAbsoluteAddress()` existe déjà pour les répertoires.
2. Comment avez vous fait pour qu'il n'y ait aucune utilisation de *instanceOf* dans votre code, alors qu'il y a plusieurs sortes de visiteurs possibles et plusieurs sortes d'élements de stockage ? Sauriez vous en dire plus à ce sujet en vous adressant à un non spécialiste qui utilise *instanceOf* ? partout.

B Composants JavaBeans et Aspects - (environ 4 points)

Plaçons nous dans le contexte de l'exemple des composants `compteur` et `afficheur` de valeur (vus en cours JavaBeans) ; composants que l'on peut tous deux mettre sur la palette dans un environnement de type *JavaBeans*.

Questions Java-Beans

1. Le code ci-dessous (listing 2)est généré automatiquement par un environnement de type *Netbeans* en réponse aux actions du programmeur pour connecté un compteur à un afficheur. Il s'agit d'un code Java Standard.

   Expliquez ce code à un non spécialiste : que fait-il ? quels schéma(s) de conception implémente-t-il ?

```
counterBean1 = new counterjavabeans.CounterBean();
afficheurBean1 = new counterjavabeans.AfficheurBean();

counterBean1.addPropertyChangeListener (
    new java.beans.PropertyChangeListener() {
        public void propertyChange(java.beans.PropertyChangeEvent evt) {
            afficheurBean1.setText(counterBean1.getValue());} }
    );
```

Listing 2 – Code généré par l'environnement *Netbeans*.

2. Quel rapport conceptuel voyez vous entre le concept de propriété liée et le schéma de conception `MVC` (*Model-View-Controller*) ?

Questions Programmation par Aspects

3. Considérez vos classes réalisant les `ElementDeStockage` (fichiers et répertoires) dans leur version originelle et dotez les d'un mécanisme de visite sans les modifier, grâce à un aspect.
4. Expliquez à un non spécialiste quel rapport il peut y avoir entre les acpects et les annotations *Java* telles que celles utilisées dans le mapping objet-relationnel de la partie II de l'examen.

C Modules et Services *Java* - Environ 3 points

Vous développez en Java-17 deux implémentations d'une interface `IService` : `ServiceA` et `ServiceB`. Vous placez chacun des services dans un module dédié, et l'interface également dans un module dédié. Le module `Client` utilise l'un ou l'autre des services. Vous cherchez maintenant à éliminer le couplage statique entre un `Client` et l'un ou l'autre des services.

Questions

1. Expliquez pourquoi il y a un tel couplage alors que les services reposent sur une interface.
2. Expliquez pourquoi utiliser un patron de conception Fabrique n'est pas une manière satisfaisante d'éliminer ce couplage.
3. Expliquez quelle est la solution basée sur les services Java qui élimine le couplage de manière satisfaisante. Donnez les grandes lignes de cette solution, sans nécessairement écrire de code mais en expliquant bien ce qui doit être mis dans le `Client` et dans les descripteurs des différents modules.

PARTIE II (10 points et 1 heure de temps estimé)

Les exemples de code sont repris directement de l'exemple applicatif exploité en TP (les salles d'enseignement à l'Université).

D Mapping Objet-Relationnel (3 points)

1. Comment résumeriez vous, en trois ou quatre phrases, l'intérêt du pattern DAO (Data Access Object) ?
2. Un extrait de la classe Campus mettant en évidence le lien entre un campus et sa collection de bâtiments est présenté.

```
1 @Entity
2 public class Campus {

4 @OneToMany(fetch = FetchType.EAGER, mappedBy="campus", cascade =
      CascadeType.ALL)
5 private List<Batiment> batiments = new ArrayList<Batiment>();
6 }
```

Listing 3 – dans la classe Campus

   - Vous direz à quoi correspondent les annotations @Entity et @OneToMany
   - Si un campus est chargé dans l'application, est ce que sa collection de bâtiments l'est aussi ?
   - Si un campus est détruit, est ce que les bâtiments qui en dépendent le sont aussi ?
3. Si l'on crée un nouvel objet Campus (via un des constructeurs de la classe) dans l'application sans chercher à le sauvegarder, dans quel état se trouve t'il ?

```
1 Campus anatomie = new Campus("Anatomie","Montpellier");
```

Listing 4 – un nouveau Campus

E Composants et injection de dépendances (5 points)

1. Quels sont les rôles dévolus aux composants de type Service ?
2. Expliquer le code (requête, retour, arguments) associé à la méthode `findByXAndY` ?

```
1 @Query("SELECT s FROM Salle s WHERE s.etage = :et and s.batiment= :bt")
2 List<Salle> findByXAndY(@Param("et") String et, @Param("bt") Batiment bt);
```

Listing 5 – dans un composant Repository

3. Quels sont les patterns qui suivent le principe d'Inversion de Contrôle (IoC) ? Expliquer en quoi ils diffèrent (les uns des autres).

4. Retrouvez les deux erreurs qui se sont glissées dans le code du listing 6 :

```
@SpringBootApplication
public class CL_Appli implements CommandLineRunner {
        private CampusService cs;
        @Autowired
        private Campus cp;
        @Autowired
        private CampusRepository cr;

       public static void main(String[] args) {
           SpringApplication.run(CL_Appli.class, args);
       }
```

Listing 6 – au niveau application

5. Donner la signification de l'annotation suivante :

```
@Autowired(required=false)
```

Listing 7 – annotation Spring

F Spring MVC (2 points)

1. Expliquer le code de la méthode de contrôleur de vue `deleteCampus` portant sur Campus et faisant appel à CampusService (objet cs)

```
@DeleteMapping("/delete/{nomC}")
public String deleteCampus(@PathVariable("nomC") String nomC) {
      cs.deleteCampus(nomC);
      return "redirect:/campuses";
   }
```

Listing 8 – méthode traitant une demande

2. Expliquer, en trois ou quatre lignes, les intérêts du pattern "Front Controller" dans Spring MVC